Design and Analysis of Algorithms CC342

Lectures 1-2

Dr. Salman Masih

Assistant Professor

Department of Computer Science

University of Management and Technology, Sialkot, Pakistan

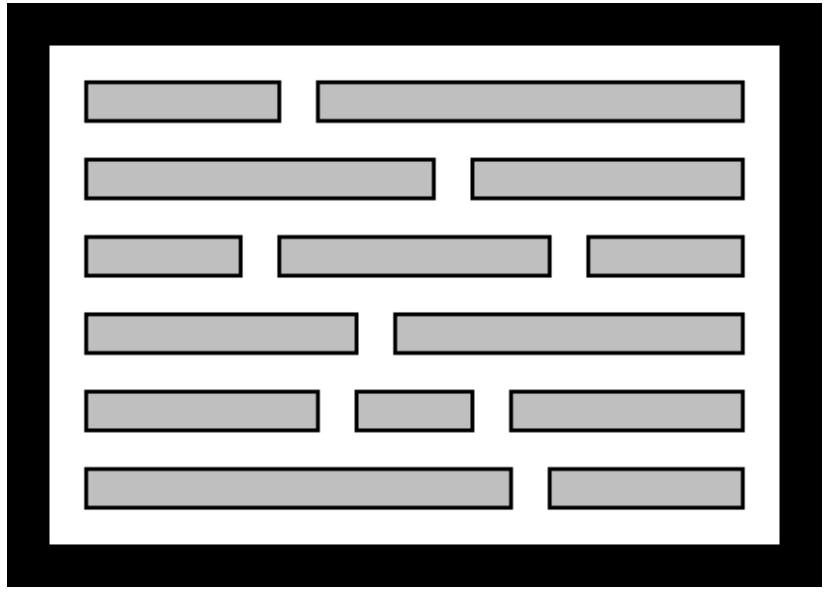
Learning Objectives

- Understand the type of problem that will be covered in this class.
- Recognize some problems for which sophisticated algorithms might not be necessary.
- Describe some artificial intelligence problems which go beyond the scope of this course.

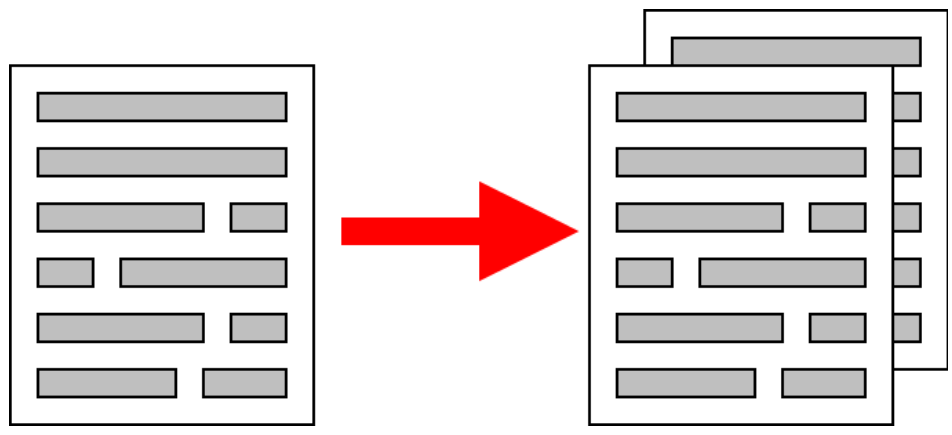
Straightforward Programming Problems

- Has straightforward implementation.
- Natural solution is already efficient.

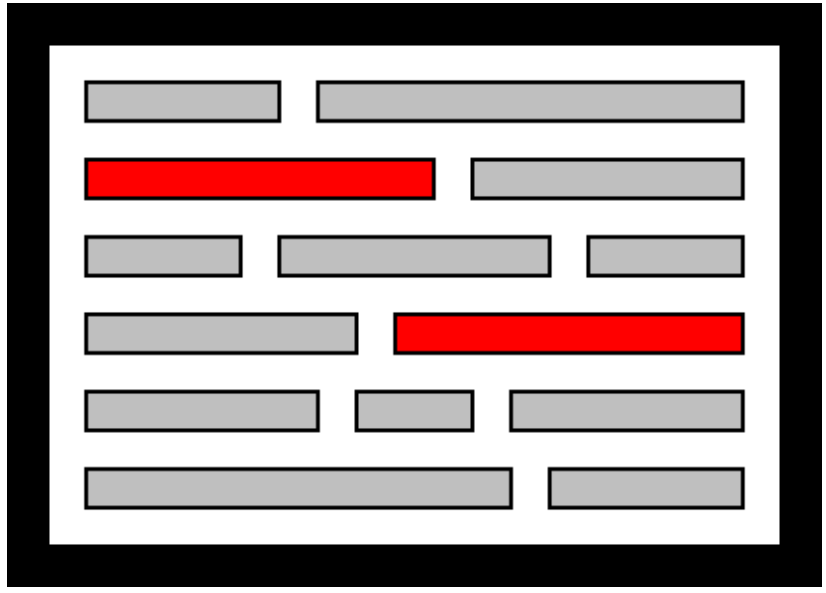
Display given text



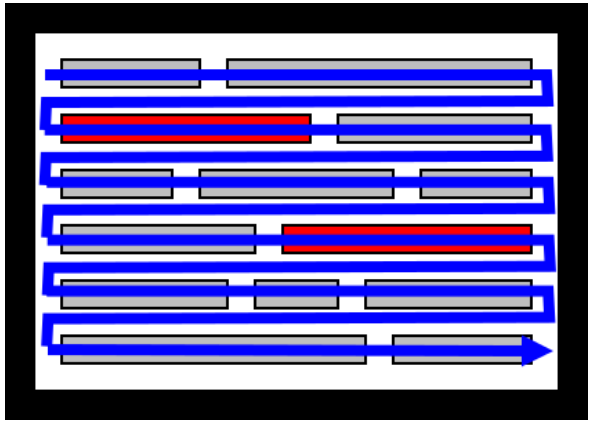
Copy a File



Search for a Given Word



Search for a Given Word



Linear Scan.

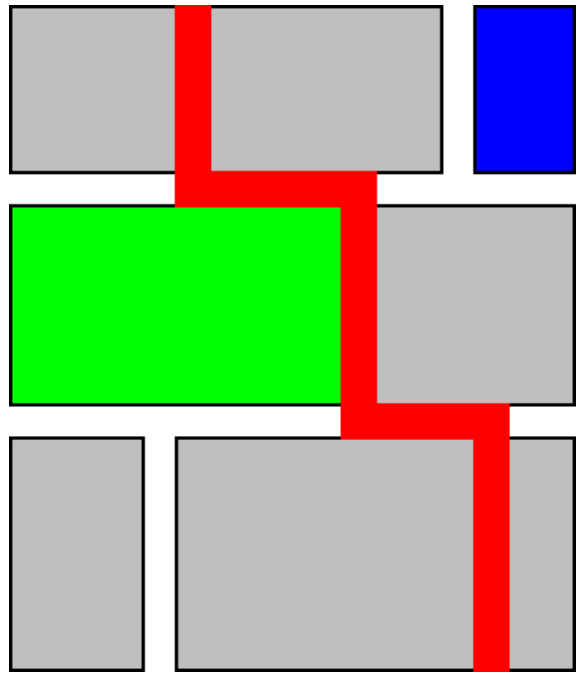
Simple Programming Problems

- Has linear scan.
- Cannot do much better.
- The obvious program works.

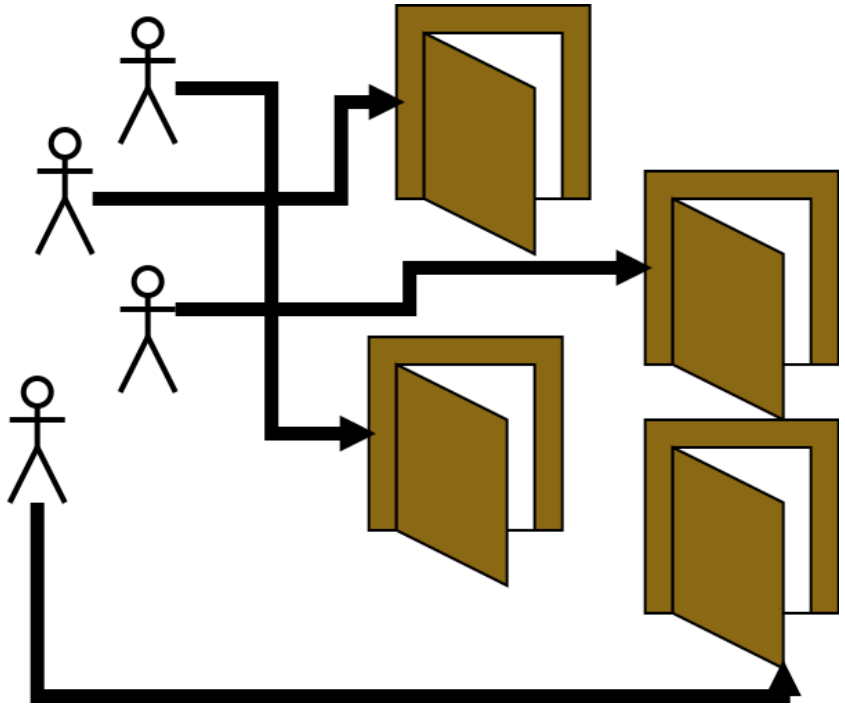
Algorithms Problems

Not so clear what to do.

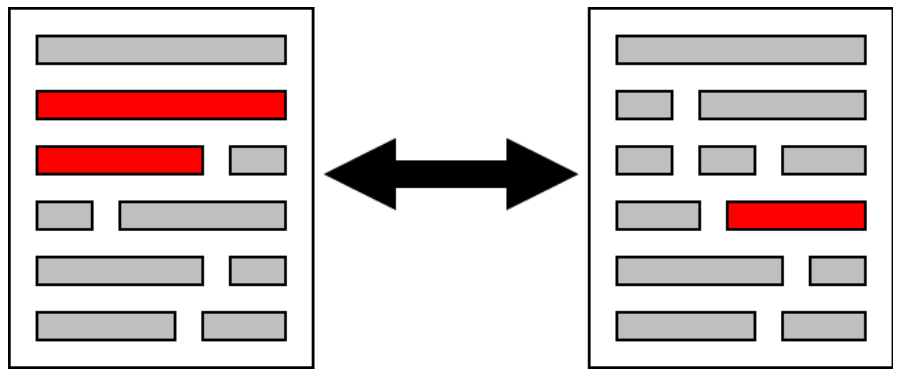
Find the Shortest Path Between Locations



Find the Best Assignment of Students to Dorm Rooms



Measure Similarity of Documents



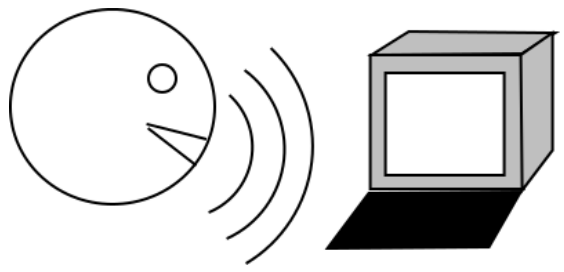
Algorithms Problems

- Not clear how to do
- Simple ideas too slow
- Room for optimization

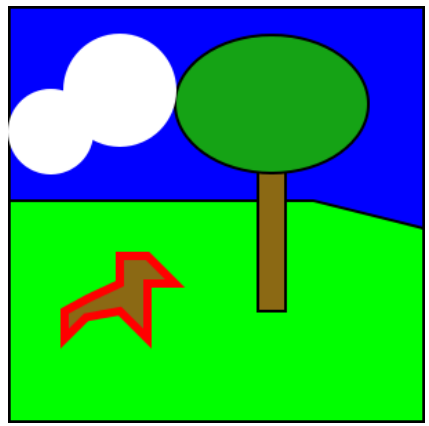
Artificial Intelligence Problems

Hard to even clearly state.

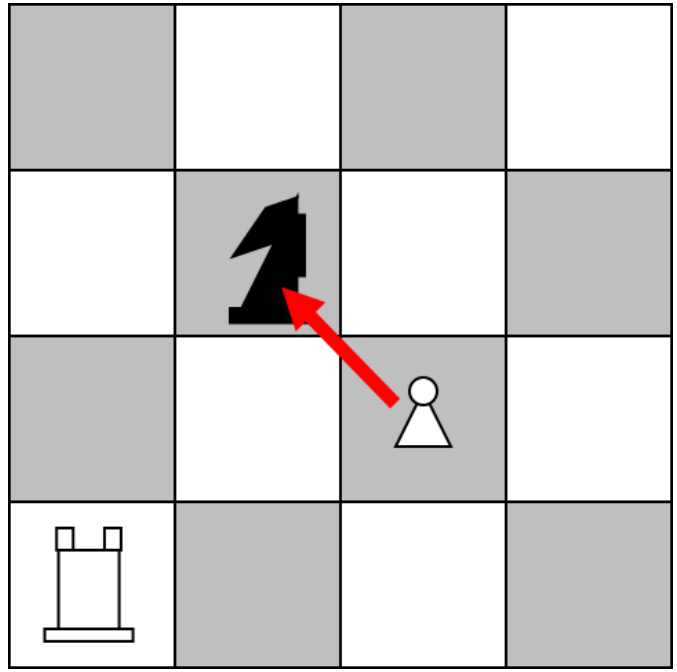
Understand Natural Language



Identify Objects In Photographs



Play Games Well



What We'll Cover

Focus on algorithms problems.

- Clearly formulated.
- Hard to do efficiently.

Learning Objectives

- Understand the definition of the Fibonacci numbers.
- Show that the naive algorithm for computing them is slow.
- Efficiently compute large Fibonacci numbers.

Outline

1. Problem Overview
2. Naive Algorithm
3. Efficient Algorithm

Definition

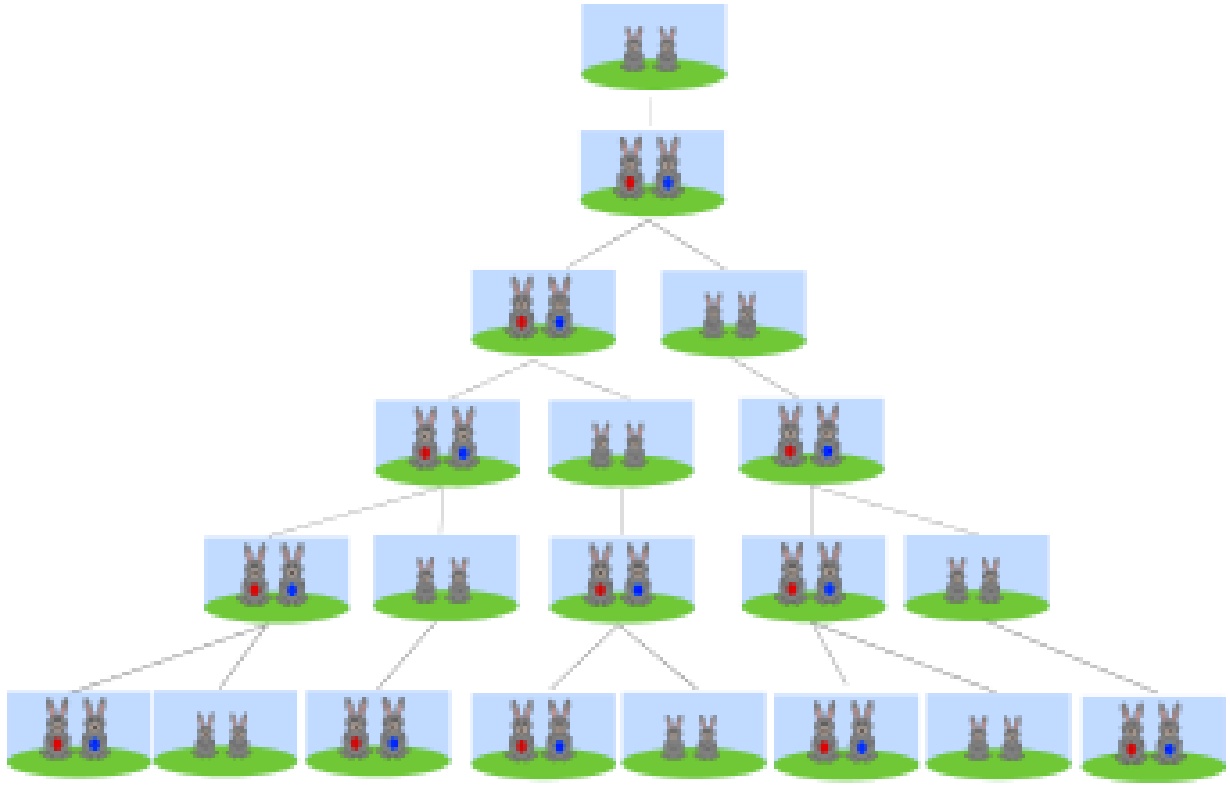
$$F_n = \begin{cases} 0, & n = 0\,, \\ 1, & n = 1\,, \\ F_{n-1} + F_{n-2}, & n > 1\,. \end{cases}$$

Definition

$$F_n = \begin{cases} 0, & n = 0\,, \\ 1, & n = 1\,, \\ F_{n-1} + F_{n-2}, & n > 1\,. \end{cases}$$

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \ldots$$

Developed to Study Rabbit Populations



Rapid Growth

Lemma

$F_n \geq 2^{n/2}$ for $n \geq 6$.

Rapid Growth

Lemma

$F_n \geq 2^{n/2}$ for $n \geq 6$.

Proof

By induction

Rapid Growth

Lemma

$F_n \geq 2^{n/2}$ for $n \geq 6$.

Proof

By induction

Base case: $n = 6, 7$ (by direct computation).

Rapid Growth

Lemma

$F_n \geq 2^{n/2}$ for $n \geq 6$.

Proof

By induction

Base case: $n = 6, 7$ (by direct computation).

Inductive step:

$$F_n = F_{n-1} + F_{n-2} \geq 2^{(n-1)/2} + 2^{(n-2)/2} \geq$$
$$2 \cdot 2^{(n-2)/2} = 2^{n/2}. \quad \square$$

Formula

Theorem

$$F_n = \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right).$$

Example

$$F_{20} = 6765$$

Example

$$F_{20} = 6765$$

$$F_{50} = 12586269025$$

Example

$$
\begin{aligned}
F_{20} &= 6765 \\
F_{50} &= 12586269025 \\
F_{100} &= 354224848179261915075
\end{aligned}
$$

Example

$$
\begin{aligned}
F_{20} &= 6765 \\
F_{50} &= 12586269025 \\
F_{100} &= 354224848179261915075 \\
F_{500} &= 139423224561697880139724382870407283950070256587697307264108962948325571622863290691557658876222521294125
\end{aligned}
$$

Computing Fibonacci numbers

Compute F_n

Input: An integer $n \geq 0$.

Output: F_n.

Outline

1. Problem Overview
2. Naive Algorithm
3. Efficient Algorithm

Algorithm

FibRecurs(n)

```
if n ≤ 1:
  return n
```

Algorithm

FibRecurs(n)

```
if n ≤ 1:
  return n
else:
  return FibRecurs(n − 1) + FibRecurs(n − 2)
```

Running time

Let $T(n)$ denote the number of lines of code executed by `FibRecurs`(n).

If $n \leq 1$

FibRecurs(n)

```
if n ≤ 1:
  return n
else:
  return FibRecurs(n − 1) + FibRecurs(n − 2)
```

If $n \leq 1$

FibRecurs(n)

```
if n ≤ 1:
  return n
else:
  return FibRecurs(n − 1) + FibRecurs(n − 2)
```

$$T(n) = 2.$$

If $n \geq 2$

FibRecurs(n)

```
if n ≤ 1:
  return n
else:
  return FibRecurs(n − 1) + FibRecurs(n − 2)
```

If $n \geq 2$

FibRecurs(n)

```
if n ≤ 1:
    return n
else:
    return FibRecurs(n − 1) + FibRecurs(n − 2)
```

$$T(n) = 3$$

If $n \geq 2$

FibRecurs(n)

```
if n ≤ 1:
  return n
else:
  return FibRecurs(n − 1) + FibRecurs(n − 2)
```

$$T(n) = 3 + T(n-1) + T(n-2).$$

Running Time

$$T(n) = \begin{cases} 2 & \text{if } n \leq 1 \\ T(n-1) + T(n-2) + 3 & \text{else.} \end{cases}$$

Running Time

$$T(n) = \begin{cases} 2 & \text{if } n \leq 1 \\ T(n-1) + T(n-2) + 3 & \text{else.} \end{cases}$$

Therefore $T(n) \geq F_n$

Running Time

$$T(n) = \begin{cases} 2 & \text{if } n \leq 1 \\ T(n-1) + T(n-2) + 3 & \text{else.} \end{cases}$$

Therefore $T(n) \geq F_n$

$T(100) \approx 1.77 \cdot 10^{21}$ (1.77 sextillion)

Running Time

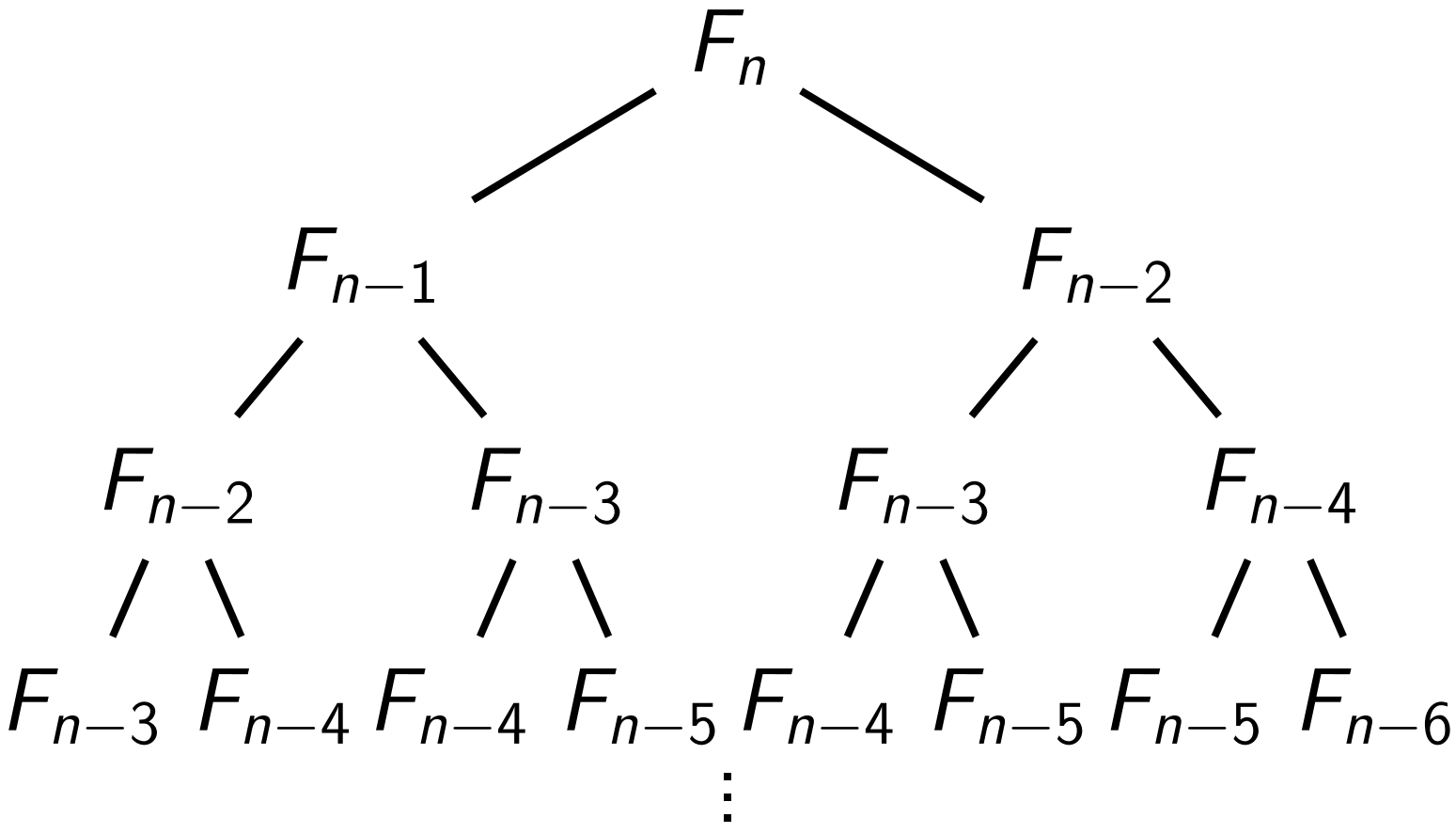
$$T(n) = \begin{cases} 2 & \text{if } n \leq 1 \\ T(n-1) + T(n-2) + 3 & \text{else.} \end{cases}$$

Therefore $T(n) \geq F_n$

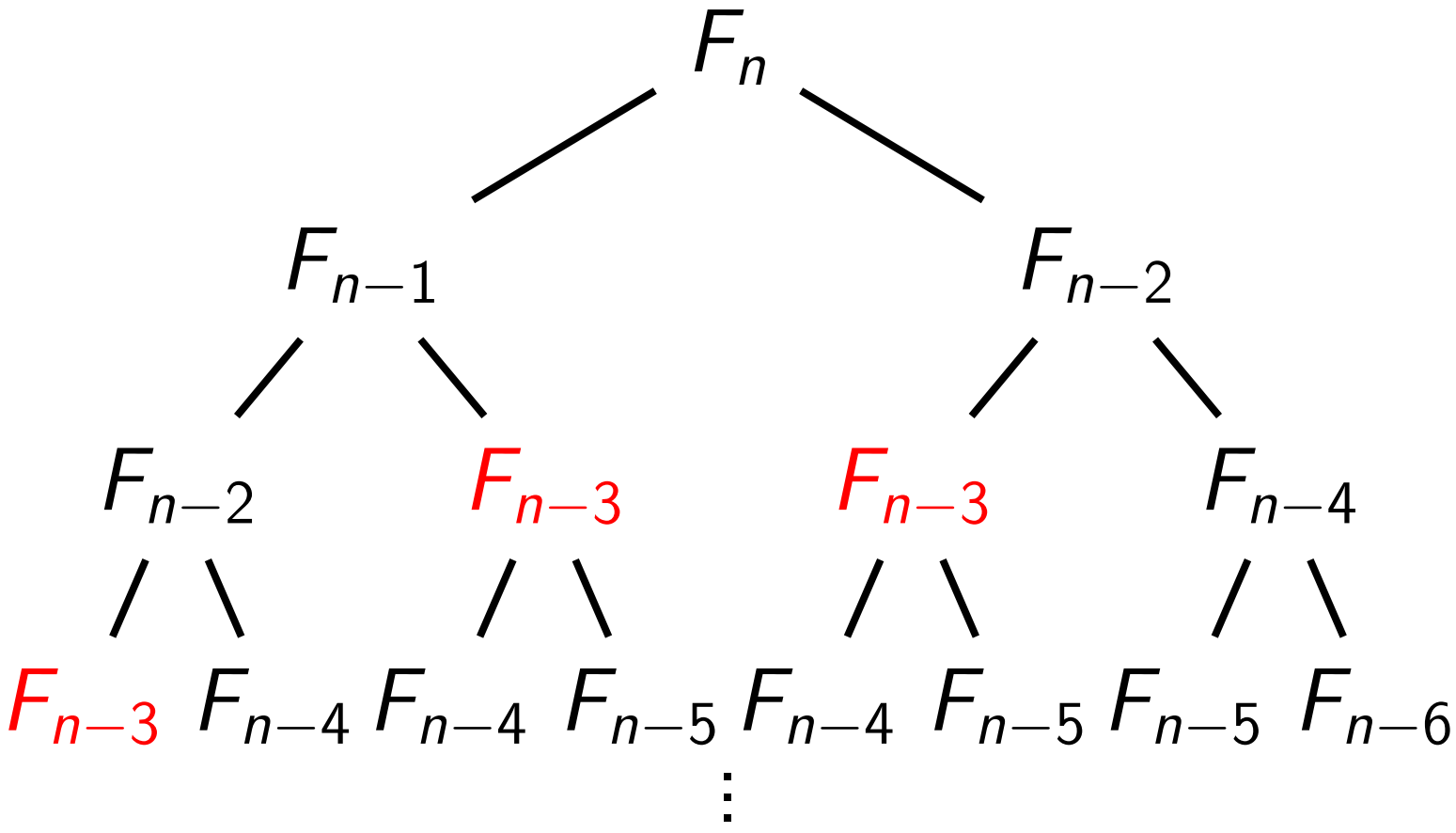
$T(100) \approx 1.77 \cdot 10^{21}$ (1.77 sextillion)

Takes 56,000 years at 1GHz.

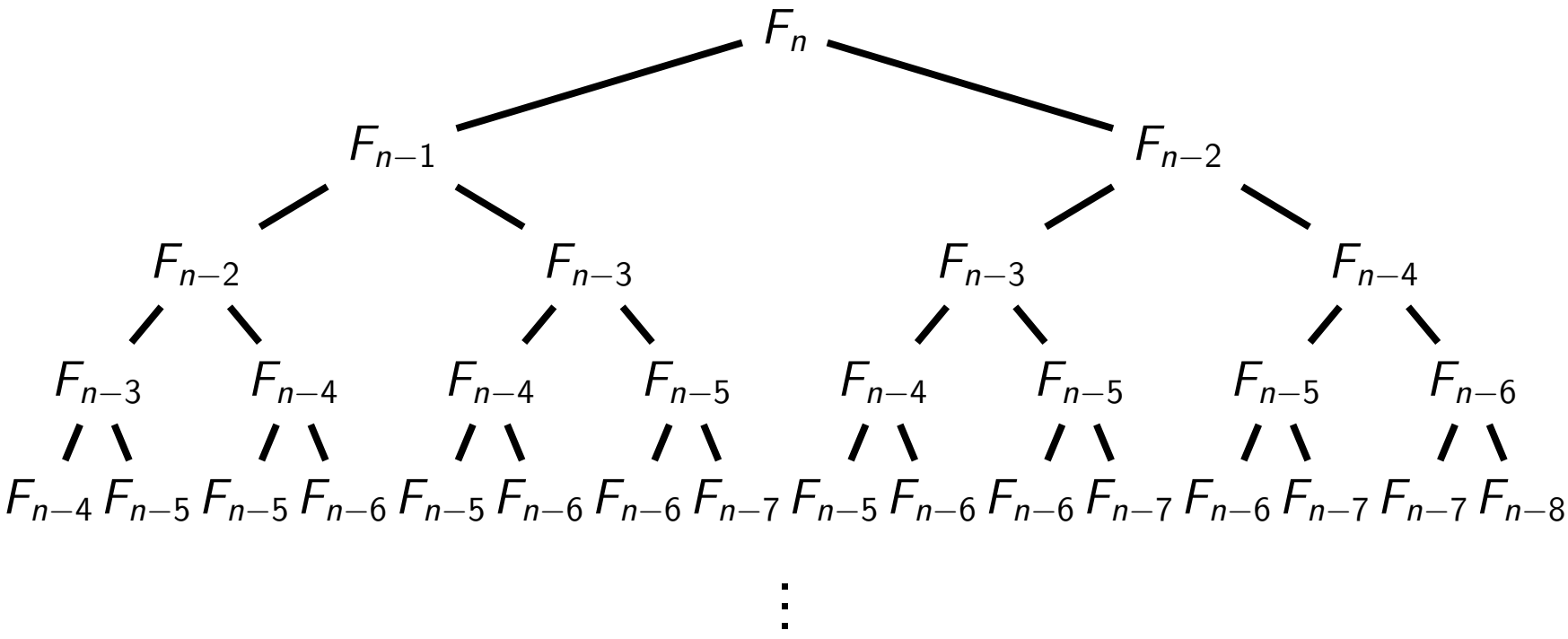
Why so slow?



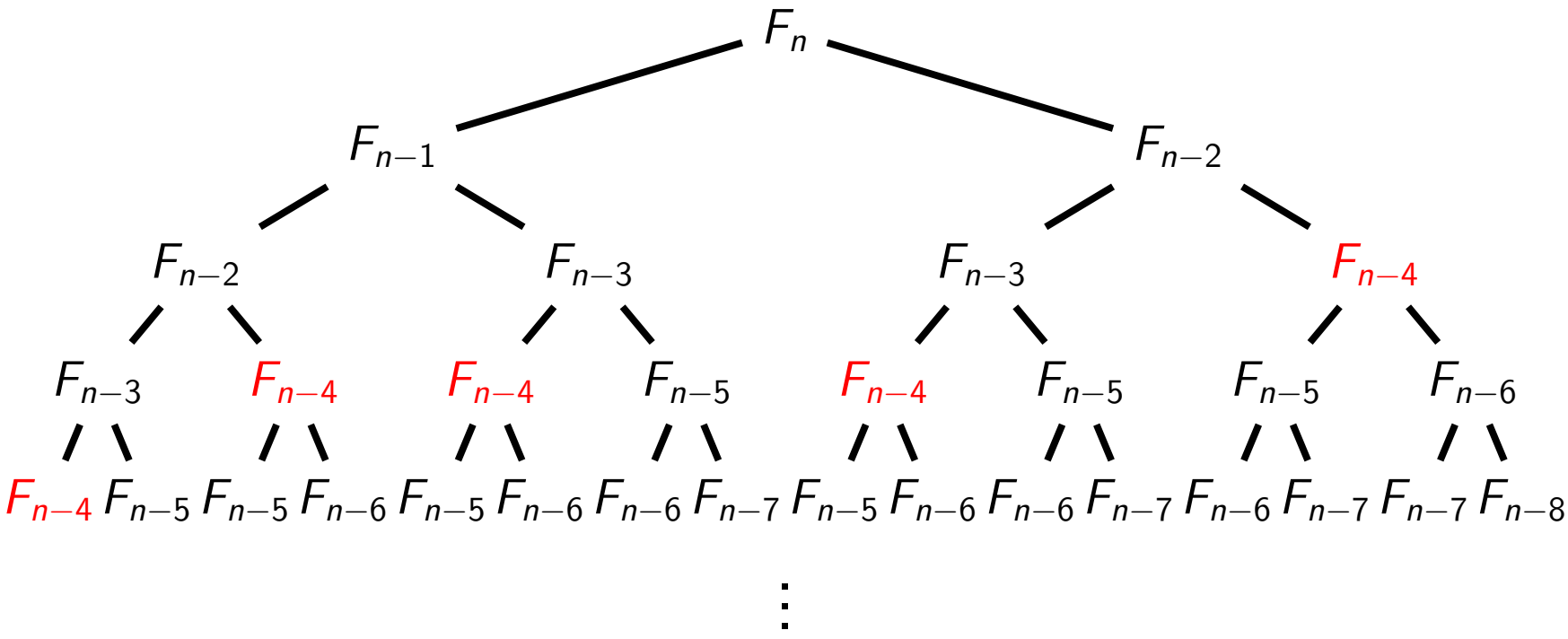
Why so slow?



Why so slow?


F_n
F_{n-1} F_{n-2}
F_{n-2} F_{n-3} F_{n-3} F_{n-4}
F_{n-3} F_{n-4} F_{n-4} F_{n-5} F_{n-4} F_{n-5} F_{n-5} F_{n-6}
F_{n-4} F_{n-5} F_{n-5} F_{n-6} F_{n-5} F_{n-6} F_{n-6} F_{n-7} F_{n-5} F_{n-6} F_{n-6} F_{n-7} F_{n-6} F_{n-7} F_{n-7} F_{n-8}
⋮

Why so slow?


F_n
F_{n-1} F_{n-2}
F_{n-2} F_{n-3} F_{n-3} F_{n-4}
F_{n-3} F_{n-4} F_{n-4} F_{n-5} F_{n-4} F_{n-5} F_{n-5} F_{n-6}
F_{n-4} F_{n-5} F_{n-5} F_{n-6} F_{n-5} F_{n-6} F_{n-6} F_{n-7} F_{n-5} F_{n-6} F_{n-6} F_{n-7} F_{n-6} F_{n-7} F_{n-7} F_{n-8}
⋮

Outline

1. Problem Overview
2. Naive Algorithm
3. Efficient Algorithm

Another Algorithm

Imitate hand computation:

0, 1

Another Algorithm

Imitate hand computation:

0, 1, 1

$0 + 1 = 1$

Another Algorithm

Imitate hand computation:

0, 1, 1, 2

$0 + 1 = 1$

$1 + 1 = 2$

Another Algorithm

Imitate hand computation:

0, 1, 1, 2, 3

$0 + 1 = 1$

$1 + 1 = 2$

$1 + 2 = 3$

Another Algorithm

Imitate hand computation:

0, 1, 1, 2, 3, 5

$0 + 1 = 1$

$1 + 1 = 2$

$1 + 2 = 3$

$2 + 3 = 5$

Another Algorithm

Imitate hand computation:

0, 1, 1, 2, 3, 5, 8

$0 + 1 = 1$

$1 + 1 = 2$

$1 + 2 = 3$

$2 + 3 = 5$

$3 + 5 = 8$

New Algorithm

FibList(n)

create an array $F[0 \ldots n]$
$F[0] \leftarrow 0$
$F[1] \leftarrow 1$
for i from 2 to n:
  $F[i] \leftarrow F[i-1] + F[i-2]$
return $F[n]$

New Algorithm

FibList(n)

```
create an array F[0...n]
F[0] ← 0
F[1] ← 1
for i from 2 to n:
    F[i] ← F[i − 1] + F[i − 2]
return F[n]
```

- $T(n) = 2n + 2$. So $T(100) = 202$.
- Easy to compute.

Summary

- Introduced Fibonacci numbers.
- Naive algorithm takes ridiculously long time on small examples.
- Improved algorithm incredibly fast.

Summary

- Introduced Fibonacci numbers.
- Naive algorithm takes ridiculously long time on small examples.
- Improved algorithm incredibly fast.

Moral: The right algorithm makes all the difference.

Learning Objectives

- Define greatest common divisors.
- Compute greatest common divisors inefficiently.

GCDs

- Put fraction $\frac{a}{b}$ in simplest form.
- Divide numerator and denominator by d, to get $\frac{a/d}{b/d}$.

GCDs

- Put fraction $\frac{a}{b}$ in simplest form.
- Divide numerator and denominator by d, to get $\frac{a/d}{b/d}$.
  - Need d to divide a and b.
  - Want d to be as large as possible.

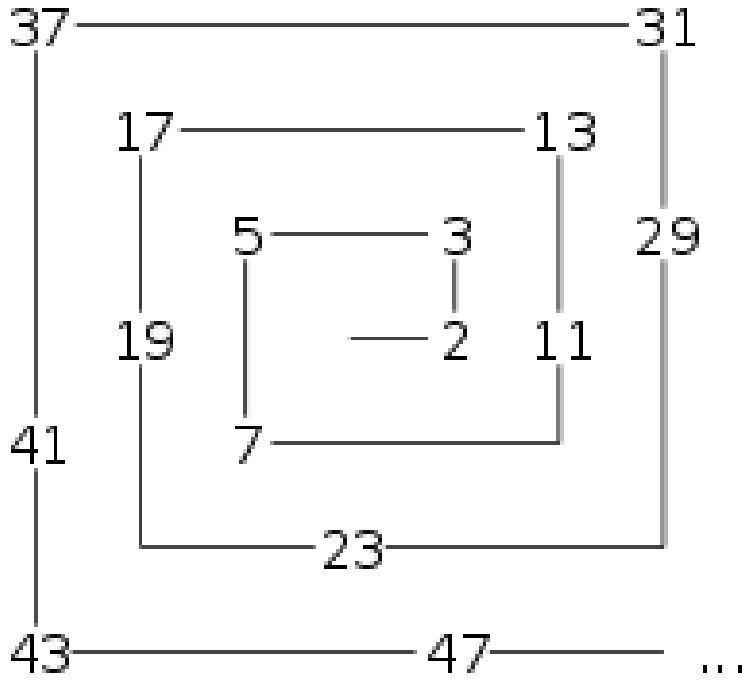
GCDs

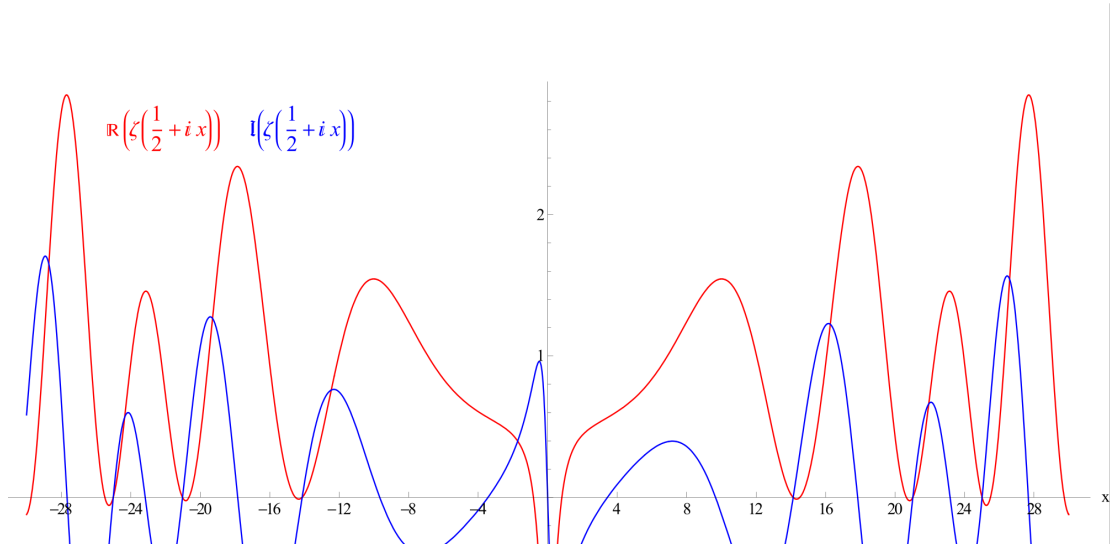
- Put fraction $\frac{a}{b}$ in simplest form.
- Divide numerator and denominator by d, to get $\frac{a/d}{b/d}$.
  - Need d to divide a and b.
  - Want d to be as large as possible.

Definition

For integers, a and b, their greatest common divisor or $\gcd(a, b)$ is the largest integer d so that d divides both a and b.

Number Theory





Cryptography



Computation

Compute GCD

Input: Integers $a, b \geq 0$.

Output: $\gcd(a, b)$.

Computation

Compute GCD

Input: Integers $a, b \geq 0$.

Output: $\gcd(a, b)$.

Run on large numbers like

$$\gcd(3918848, 1653264).$$

Naive Algorithm

Function NaiveGCD(a, b)

$best \leftarrow 0$
for d from 1 to $a + b$:
    if $d | a$ and $d | b$:
        $best \leftarrow d$
return $best$

Naive Algorithm

Function NaiveGCD(a, b)

```
best ← 0
for d from 1 to a + b:
    if d|a and d|b:
        best ← d
return best
```

- Runtime approximately $a + b$.
- Very slow for 20 digit numbers.

Learning Objectives

- Implement the Euclidean Algorithm.
- Approximate the runtime.

GCDs

Definition

For integers, a and b, their greatest common divisor or $\gcd(a, b)$ is the largest integer d so that d divides both a and b.

GCDs

Definition

For integers, a and b, their greatest common divisor or $\gcd(a, b)$ is the largest integer d so that d divides both a and b.

Compute GCD

Input: Integers $a, b \geq 0$.

Output: $\gcd(a, b)$.

Key Lemma

Lemma

Let a' be the remainder when a is divided by b, then

$$\gcd(a, b) = \gcd(a', b) = \gcd(b, a').$$

Proof

Proof (sketch)

- $a = a' + bq$ for some q
- d divides a and b if and only if it divides a' and b

Euclidean Algorithm

Function EuclidGCD(a, b)

```
if b = 0:
    return a
a' ← the remainder when a is
     divided by b
return EuclidGCD(b, a')
```

Euclidean Algorithm

Function EuclidGCD(a, b)

```
if b = 0:
   return a
a' ← the remainder when a is
     divided by b
return EuclidGCD(b, a')
```

Produces correct result by Lemma.

Example

$$\gcd(3918848, 1653264)$$

Example

$$\begin{aligned} & \gcd(3918848, 1653264) \\ = & \gcd(1653264, 612320) \end{aligned}$$

Example

$$\begin{aligned} & \gcd(3918848, 1653264) \\ = & \gcd(1653264, 612320) \\ = & \gcd(612320, 428624) \end{aligned}$$

Example

$$
\begin{aligned}
& \gcd(3918848, 1653264) \\
= \; & \gcd(1653264, 612320) \\
= \; & \gcd(612320, 428624) \\
= \; & \gcd(428624, 183696)
\end{aligned}
$$

Example

$$
\begin{aligned}
&\gcd(3918848, 1653264) \\
=\ &\gcd(1653264, 612320) \\
=\ &\gcd(612320, 428624) \\
=\ &\gcd(428624, 183696) \\
=\ &\gcd(183696, 61232)
\end{aligned}
$$

Example

$$
\begin{aligned}
& \gcd(3918848, 1653264) \\
= \gcd(1653264, 612320) \\
= \gcd(612320, 428624) \\
= \gcd(428624, 183696) \\
= \gcd(183696, 61232) \\
= \gcd(61232, 0)
\end{aligned}
$$

Example

$$
\begin{aligned}
&\gcd(3918848, 1653264)\\
=&\gcd(1653264, 612320)\\
=&\gcd(612320, 428624)\\
=&\gcd(428624, 183696)\\
=&\gcd(183696, 61232)\\
=&\gcd(61232, 0)\\
=&61232.
\end{aligned}
$$

Runtime

- Each step reduces the size of numbers by about a factor of 2.
- Takes about $\log(ab)$ steps.

Runtime

- Each step reduces the size of numbers by about a factor of 2.
- Takes about $\log(ab)$ steps.
- GCDs of 100 digit numbers takes about 600 steps.
- Each step a single division.

Summary

- Naive algorithm is too slow.
- The correct algorithm is much better.
- Finding the correct algorithm requires knowing something interesting about the problem.